Ders 24 Genişletilmiş Notu: Metin Arama: Kaba Kuvvet, Boyer-Moore ve Rabin-Karp

Ders 24 Genişletilmiş Notu: Metin Arama: Kaba Kuvvet, Boyer-Moore ve Rabin-Karp

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Metin Arama Problemi ve Kullanım Alanları
- 2. Gösterim
- 3. Kaba Kuvvet (Brute Force) Yöntemi
- 4. Daha Akıllı Kaydırma Fikri
- 5. Boyer-Moore Ailesi: Ön İşlem ve Sağdan Sola Karşılaştırma
- 6. Boyer-Moore-Horspool Algoritması
- 7. Boyer-Moore: Kötü Karakter Kaydırması
- 8. Boyer-Moore: İyi Sonek Kaydırması
- 9. İki Kaydırmanın Birleştirilmesi
- 10. Tablet Üzerinde Örnek: BAOBAB
- 11. Rabin-Karp Algoritması
- 12. Karmaşıklık Karşılaştırması ve Deneysel Sonuçlar
- 13. Algoritmaların C ile Gerçeklemesi
- Örnek Sorular
- Dersin Toparlaması

1. Metin Arama Problemi ve Kullanım Alanları

Dönemin son iki haftasına girilirken bu haftanın konusu metin arama (string matching) algoritmalarıydı. Hoca, önümüzdeki hafta sıralama (sorting) algoritmalarının karşılaştırmalı olarak değerlendirileceğini söyledi.

Metin aramaya örüntü eşleme (pattern matching) ya da örüntü arama (pattern search) da denir. Bir metnin (text) içinde, karakterlerden oluşan bir örüntünün (pattern) geçip geçmediği, geçiyorsa kaç kez ve nerelerde geçtiği bulunmaya çalışılır. Birden fazla karakterin bir araya geldiği yapıya karakter dizisi (string) diyoruz. Bazı dillerde bu bir veri tipi olarak tanımlıdır. Hoca örnek olarak Java'da `String` tipinin olduğunu, C'de ise ayrı bir string tipinin kullanılmadığını söyledi.

> **Ek bilgi (derste anlatılmadı):** C dilinde metin, sonunda `'\0'` (null) karakteri bulunan bir `char` dizisi olarak tutulur. `strlen` gibi kütüphane fonksiyonları uzunluğu bu sonlandırıcı karaktere kadar sayarak bulur.

Hocanın saydığı kullanım alanları şunlardır:

- **Kelime işlemciler:** Word belgelerinde bir kelimeyi ya da ifadeyi aratıp geçtiği yerleri bulmak, sonra o bölümü okumak ya da bul/değiştir ile güncellemek. Yüzlerce sayfalık bir metinde aramada zaman karmaşıklığı çok daha önemli hâle gelir.
- **Belge benzerliği:** Tek bir kelime değil, çok sayıda örüntünün başka bir metinde geçip geçmediğine bakılır. Kod benzerliği ve tezlerin özgünlük denetimi bu gruptadır. On binlerce, yüz binlerce, milyonlarca metin tarandığında karmaşıklık daha da önem kazanır.

- **Biyoenformatik:** DNA dizilimlerinde arama.
- **Veri madenciliği:** Belirli bir örüntü varsa ilgili kısmın ortaya çıkarılması.
- **Metin sınıflandırma** ve, liste genişletildiğinde, konuşma tanımaya kadar giden uygulamalar.

Bu derste daha basit bir durum, yani iki karakter dizisi arasındaki örüntü arama algoritmaları ele alındı. Hoca, bu algoritmalardan büyük metinlere geçişin kolayca yapılabileceğini söyledi.

2. Gösterim

Slayttaki gösterim şöyledir:

- T: metin (text), uzunluğu n.
- P: örüntü (pattern), uzunluğu m.
- Σ: alfabe, yani kullanılabilecek bütün sembollerin kümesi.
- Tek harfler a, b, c gibi küçük harflerle, karakter dizileri x, y, z gibi harflerle gösterilir.

Pratikte metin örüntüden çok daha uzundur: $n \gg m$.

3. Kaba Kuvvet (Brute Force) Yöntemi

3.1 Çalışma Mantığı

İlk akla gelen yaklaşım, naif metin arama (naive string search) da denen kaba kuvvet yöntemidir. Örüntü metnin başına hizalanır ve karakterler tek tek karşılaştırılır. Eşleşme sağlanamazsa örüntü bir sağa kaydırılır ve karşılaştırma yeniden yapılır. Bu, metnin geri kalanında eşleşme bulunana ya da metin bitene kadar sürer.

Hoca bu yöntemin ilk dönemlerde ayrıştırıcılarda (parser), spam filtrelerinde, kelime işlemcilerde ve web arama motorlarında yoğun olarak kullanıldığını söyledi.

Slayttaki ilk örnekte örüntü metin boyunca birer birer kaydırılıyor: önce `N` ile `H` karşılaştırılıyor, eşit değil, bir kaydırılıyor; sonra `N` ile `A`, yine eşit değil, bir kaydırılıyor. Bir noktada `N`, `E`, `E`, `D` eşleşiyor, fakat son harf tutmadığı için örüntü yine bir sağa kaydırılıyor.

3.2 Kod

Slayttaki `search` metodu iç içe iki döngüden oluşur:

```
int search(char *pat, char *txt) {
  int M = strlen(pat);
  int N = strlen(txt);
  int i, j;
  for (i = 0; i <= N - M; i++) {
    for (j = 0; j < M; j++)
      if (txt[i + j] != pat[j])
        break;
    if (j == M)
      return i;       /* eşleşme bulundu */
  }
  return N;           /* bulunamadı */
}
```

- **Dış döngü** metin üzerinde ilerler, ama metnin sonuna kadar değil, $N - M$ konumuna kadar gider. Hoca bunu şöyle açıkladı: Bu noktadan sonra örüntü metne sığmaz. Bir karakter eşleşse bile örüntünün tamamının eşleşme ihtimali yoktur, bu yüzden sonrasına bakmaya gerek yoktur.
- **İç döngü** örüntünün boyu kadar döner.
- İç döngü kesilmeden $j = M$ olursa bütün karakterler eşleşmiştir ve konum i döndürülür.

3.3 En Kötü Durum

En kötü durumda bu yöntemin karmaşıklığı $O(nm)$'dir. Hoca bu durumun, örüntünün tekrar eden sembollerden oluştuğu durumlarda ortaya çıktığını söyledi. Biyoenformatikten bir örnekte `GCT` örüntüsü aranıyordu. Hoca, örüntü `GCT` yerine `AAB` ya da `ABABAB` gibi tekrarlı bir örüntü olursa ve metinde de bu örüntüye çok sık rastlanırsa, her konumda neredeyse son bir iki sembole kadar karşılaştırma yapılacağını anlattı. Son semboller tutmadığı için her seferinde bir sağa gidilir ve $n \cdot m$'lik karmaşıklık oluşur.

Slayttaki en kötü durum örneği: `AAAAAAAAAB` metninde `AAAAB` aranıyor. Her hizalamada dört `A` eşleşiyor, beşinci karakterde `A` ile `B` eşit olmadığı için örüntü bir kaydırılıyor. Sayımı tamamlayalım (ara adım eklendi):

- Metin uzunluğu $n = 10$, örüntü uzunluğu $m = 5$. Hizalama sayısı $n - m + 1 = 6$.
- İlk 5 hizalamanın her birinde 4 eşleşme + 1 uyuşmazlık = 5 karşılaştırma: $5 \cdot 5 = 25$.
- 6. hizalamada (konum 5) beş karakter de eşleşir: 5 karşılaştırma.
- Toplam: $25 + 5 = 30$ karakter karşılaştırması. Bu, $(n-m+1) \cdot m = 6 \cdot 5 = 30$ üst sınırına tam eşittir.

Hoca, örüntü uzadıkça bu karmaşıklığın "işin içinden çıkılmaz" hâle geldiğini vurguladı. Bu yüzden tekrar eden yapıya sahip örüntü ve metinler için kaba kuvvet uygun bir yaklaşım değildir.

4. Daha Akıllı Kaydırma Fikri

Hoca öğrencilere sordu: Örüntüyü birer birer kaydırmak maliyetli görünüyor; elinizde olsaydı nasıl kaydırırdınız?

Slayttaki örnekte `ABRA` örüntüsü aranıyordu. İlk `A`'lar ve `B`'ler eşleşiyor, ama sonraki karakter tutmuyor. Bir öğrenci, örüntünün bir sonraki `A` harfinin altına kaydırılması gerektiğini söyledi. Hoca bu fikri genelleştirdi:

- Bir uyuşmazlık yakalandığında örüntüyü her koşulda bir sağa kaydırmak yerine, mümkünse örüntü boyu kadar, mümkün değilse eşleşmenin sağlanabileceği ilk noktaya kadar kaydırmak gerekir.
- `ABRA`'yı `B`'nin altına kaydırmanın anlamı yoktur, orada eşleşme ihtimali yoktur. En iyi ihtimalle örüntünün `A`'sı metindeki bir sonraki `A`'nın altına gelmelidir ki sonraki karakterlere bakılsın.
- `ABCD` metninde `XYZ` aranıyorsa, `X` ile `A` eşleşmediği anda `XYZ` doğrudan metnin sonrasına konumlandırılabilir. Çünkü `X` metinde hiç geçmiyor.

Bu yüzden hangi karakterlerin nerede bulunduğunu önceden bilmek, kaydırmayı en iyi hâle getirmek için önemlidir. Hoca, kaba kuvvet yönteminin asıl sorununu şöyle anlattı: Eşleşme ihtimali olmayan birçok seçeneği de birer kaydırarak denemeye devam ediyoruz. Önceden bir işlem yapmadığımız için bunu bilmiyoruz ve "körebe gibi" gözümüzü kapatıp tek tek deniyoruz. Bu akıllıca bir çözüm değildir.

5. Boyer-Moore Ailesi: Ön İşlem ve Sağdan Sola Karşılaştırma

Boyer-Moore algoritması bu fikri performans olarak en iyi yansıtan algoritmalardan biridir. Tek dezavantajı **ön işlem (preprocessing)** gerektirmesidir.

> **Hocanın vurgusu:** Ön işlem aranan metin üzerinde değil, örüntü üzerinde yapılır. On bin satırlık, on bin kelimelik metni değil; beş, on, elli sembollük örüntüyü ön işlemden geçirirsiniz. m, n'den çok küçük olduğu için bu büyük bir dezavantaj yaratmaz.

Örüntü büyüdükçe ön işlem biraz daha yavaş çalışır. Ama aynı ön işlem, özellikle tekrar etmeyen örüntülerde aramanın çok daha hızlı yapılmasını sağlar. Bu yüzden örüntü boyunun büyümesi bir dezavantaj gibi görünse de Boyer-Moore'un performansı örüntü büyüdükçe artar. Hoca bunu dersin sonunda bir grafikle gösterdi (Bölüm 12).

Boyer-Moore, işe yaramayacak seçenekleri eleme fikrini, örüntü ile metni **sondan hizalayarak** ve karakterleri sağdan sola karşılaştırarak uygular. Knuth-Morris-Pratt (KMP) algoritması

benzer bir mantık kullanır ama karşılaştırmayı **baştan**, soldan sağa yapar. İkisi arasındaki en temel fark budur. İkisi de metin ile örüntü arasında o ana kadar oluşan eşleşme bilgisini kullanarak karar verir.

Boyer-Moore iki yaklaşımla anlatıldı:

- **Boyer-Moore-Horspool:** Daha kolay olduğu için önce bu anlatıldı.
- **Boyer-Moore:** Esas algoritma; iki ayrı kaydırma kuralı kullanır.

6. Boyer-Moore-Horspool Algoritması

6.1 Dört Durum

Metnin $s_0 \dots s_{n-1}$ olduğunu ve ortasında bir yerde `BARBER` örüntüsünü ($m = 6$) aradığımızı düşünelim. Karşılaştırma örüntünün son karakterinden başlar. Örüntünün son karakterinin hizasındaki metin karakterine c diyelim. Kaydırma miktarı c'ye göre belirlenir.

Durum 1: c örüntüde yok. Örnekte c = `S`, örüntünün son harfi `R` ile eşleşmiyor. Örüntü bir değil, boyu kadar, yani 6 sağa kaydırılır. Hoca neden bunu "gönül rahatlığıyla" yapabildiğimizi sordu. Bir öğrencinin sohbet kutusuna yazdığı cevap: `S` harfi `BARBER` içinde geçmediği için onunla hizalanabilecek bir harf yoktur. Altı kez birer kaydırmak yerine bir seferde altı kaydırmak çok ciddi bir avantaj sağlar.

Durum 2: c örüntüde var. Örnekte c = `B` ve son harfle uyuşmuyor. Bu kez örüntü, içindeki **en sağdaki** (son karakter hariç) `B` metindeki `B` ile hizalanacak şekilde kaydırılır. `BARBER`'da sağdan bakınca ilk `B` sondan iki önceki konumdadır, bu yüzden kaydırma 2'dir. Hoca devamını da söyledi: c = `R` olsaydı kaydırma 3, c = `A` olsaydı 4 olurdu. Hoca, neden soldaki `B` ile değil de sağdaki ile hizaladığımızı da sordu. Cevabı: Metindeki eşleşmeyi bozan harfin örüntüde olup olmadığına bakılır; varsa örüntüde sağdan en yakın olan geçişi alınır.

Durum 3: Eşleşme var ama c örüntüde başka yerde yok. Örnekte örüntü `LEADER`, metinde `...MER` var: `R`'ler eşleşmiş, `E`'ler eşleşmiş, `M` ile `D` eşleşmemiş. Örüntü yine komple, boyu kadar kaydırılıyor. Hocanın sorusu: Bunu neden yapıyoruz, `M`'nin `D` ile eşleşmemesi mi, başka bir şey mi? Bir öğrencinin cevabı: Örüntüde başka `R` yok. Kaydırma hep son karakter hizasındaki c'ye (burada `R`'ye) göre belirlenir. `LEADER`'ın ilk beş karakterinde `R` olmadığı için `R` ile tekrar hizalanma ihtimali kalmamıştır.

Durum 4: Eşleşme var ve c örüntüde başka yerde de var. Örnekte örüntü `REORDER`: `R`'ler eşleşmiş, `A` ile `E` eşleşmemiş. Eşleşmeyen `A` karakterine bakıp ilerleseydik, `A` örüntüde olmadığı için örüntüyü boyu kadar kaydırırdık. Ama Horspool'da c uyuşmayan karakter değil, son karakter hizasındaki `R`'dir. Örüntüde başka `R` olduğu için o `R` ile hizalanınca hâlâ eşleşme ihtimali vardır. Örüntü ilk görülen (sağdan) `R`'ye kadar kaydırılır. Böylece bir eşleşmeyi gözden kaçırma olasılığı ortadan kalkar.

6.2 Kaydırma Tablosu

Ön işlemde bir başvuru tablosu (lookup table), yani **kaydırma tablosu (shift table)** oluşturulur. Bu tablo, metinde olabilecek her sembol için kaydırma miktarını saklar. Slayttaki tablo `BARBER` içindir:

Karakter	A	B	C	D	E	F	…	R	…	Diğerleri
Kaydırma	4	2	6	6	1	6	…	3	…	6

Hoca bu değerlerin nasıl elde edildiğini derste soru-cevapla kurdu:

- **Başlangıç değeri:** Tablonun her hücresine örüntü uzunluğu m yazılır. `BARBER` için 6. Çünkü bir harf örüntüde yoksa örüntü boyu kadar kaydırılır. `C`, `D`, `F` gibi harfler için 6 bu yüzden görülür.
- **Örüntü üzerinde soldan sağa gezilir** ve j konumundaki karaktere $m - 1 - j$ yazılır (karakterin sondan uzaklığı):
  - `B` ($j = 0$): $6 - 1 - 0 = 5$.
  - `A` ($j = 1$): 4.

  - `R` ($j = 2$): 3.
  - `B` ($j = 3$): 2. Önceki 5'in üzerine yazılır.
  - `E` ($j = 4$): 1.
- **Son karakter işlenmez.** Örüntünün son karakteri `R` ($j = 5$) için 0 yazılmaz; `R`'nin değeri 3 olarak kalır.

Hoca son karakterin neden işlenmediğini şöyle açıkladı: Karşılaştırma sağdan sola yapılır ve son karakter her hizalamada zaten karşılaştırmaya girer. Bu yüzden son harfin tabloya bir etkisi olmaz. Kural, örüntünün başından sonundaki harften bir öncekine kadar gitmektir.

> **Hocanın vurgusu:** İlk `B` görüldüğünde tabloya 2 değil 5 yazılır. Bilgisayar örüntüyü tararken ilerideki ikinci `B`'yi henüz görmez. Örüntü üzerinde ilerledikçe değer güncellenir; başka `B` olmasaydı değer 5 kalırdı.

Soru: Ön işlemde sıfırdan başlayıp sağdan saymak daha mantıklı olmaz mı? **Cevap:** O zaman ilk gördüğümüz `B`'ye 2 yazdıktan sonra, daha soldaki `B`'yi gördüğümüzde doğru değerin üzerine yanlış değeri yazmış oluruz. Soldan sağa gidince aynı karakterin en sağdaki geçişi önceki değeri ezer ve sonuç doğru çıkar. Aksi hâlde hatalı çıktı üretilir.

Tablo örüntü kısa olduğu için çok düşük maliyetle oluşturulur.

```
ALGORITHM ShiftTable(P[0..m-1])
  // Girdi: örüntü P[0..m-1] ve olası karakterlerin alfabesi
  // Çıktı: alfabe boyunda, kaydırma miktarlarını tutan Table
  for i <- 0 to size-1 do Table[i] <- m
  for j <- 0 to m-2 do Table[P[j]] <- m - 1 - j
  return Table
```

6.3 Eşleştirme Sözde Kodu

Hoca eşleştirme kısmının çok basit bir kodu olduğunu söyledi. İki `while` döngüsü vardır. Dış döngü metnin sonuna gelinip gelinmediğini, iç döngü örüntüyle eşleşmenin ne oranda sağlandığını denetler.

```
ALGORITHM HorspoolMatching(P[0..m-1], T[0..n-1])
  ShiftTable(P[0..m-1])
  i <- m - 1                  // örüntünün sağ ucunun metindeki konumu
  while i <= n - 1 do
    k <- 0                    // eşleşen karakter sayısı
    while k <= m - 1 and P[m-1-k] = T[i-k] do
      k <- k + 1
    if k = m
      return i - m + 1        // eşleşmenin başladığı konum
    else i <- i + Table[T[i]]
  return -1
```

- İç döngüye girmeden k sıfırlanır.
- $k \leq m-1$ olduğu ve karakterler birebir eşleştiği sürece sondan başa doğru karşılaştırılır. `BARBER`'da önce `R`, sonra `E`, sonra `B` … en başa kadar gelinir. Bu sırada k artırılır.
- $k = m$ olursa eşleşmenin sağlandığı konum döndürülür.
- İç döngüde k, m'e ulaşamazsa `else` koluna girilir. Kaydırma miktarı tablodan $T[i]$ (son karakter hizasındaki metin karakteri) için okunur ve i'ye eklenir. Böylece örüntü ya en fazla boyu kadar ya da içindeki bir harf metinle hizalanacak kadar kaydırılır.

7. Boyer-Moore: Kötü Karakter Kaydırması

7.1 Neden İki Kural?

Esas Boyer-Moore algoritmasında iki kaydırma kuralı vardır: **kötü karakter kaydırması (bad symbol shift)** ve **iyi sonek kaydırması (good suffix shift)**. Horspool tek bir başvuru tablosuna bakarak karar veriyordu. Boyer-Moore'daki soru şudur: Örüntünün içinde geçen tekrar eden parçaların bilgisini de kullanırsak daha iyi bir sonuç elde eder miyiz?

Hoca önce `NEEDLE` örüntüsüyle (slayttaki iğne-saman yığını örneği) birkaç durumu gösterdi:

- T ile D eşleşmiyor (E ve L eşleşmiş). T örüntüde olmadığı için örüntü T'nin sonrasına kaydırılıyor.
- Uyuşmayan karakter N ise, N örüntüde var. Örüntünün N'si metnin N'si ile hizalanacak şekilde kaydırılıyor.
- Uyuşmayan karakter E ise ve en sağdaki E ile hizalamaya kalkarsak, bu örüntünün geri, yani sola alınması anlamına gelir. Slaytta bu duruma bir soru işareti konmuş. Bu durumda örüntü bir sağa kaydırılıyor. Hoca bunun da bir sezgisel (heuristic) yaklaşım olduğunu söyledi.

7.2 Formül

BARBER örneğinde metin `... S E R ...` biçimindedir: R'ler ve E'ler eşleşmiş, S eşleşmemiş. Slaytta iki örnek hesap vardı:

- Uyuşmayan karakter S: $t_1(S) - 2 = 6 - 2 = 4$.
- Uyuşmayan karakter A: $t_1(A) - 2 = 4 - 2 = 2$.

Burada t_1, Horspool'daki kaydırma tablosunun aynısıdır. Bir öğrenci formülü sözle şöyle anlattı: S kaydırma tablosunda 6'ydı; S örüntüde olmadığı için 6 kaydırıyorduk. Ama iki karakter zaten eşleştiği için 4 kaydırılıyor. Yani tablodaki değerden eşleşen karakter sayısı çıkarılıyor.

Genel formül:

$$d_1 = \max\left(t_1(c) - k,\ 1\right)$$

- c: metindeki **uyuşmayan** karakter.
- k: uyuşmazlıktan önce eşleşen karakter sayısı.

A örneğinde A örüntüde bulunduğu ve son konumuna uzaklığı 4 olduğu için $4-2 = 2$ kaydırma yapılır.

7.3 Neden "virgül 1"?

Hoca formüldeki $\max(\ldots, 1)$'in sebebini sordu. Öğrencilerin cevabı: Kaydırma en az 1 olmalı. Hoca açıkladı: $t_1(c) - k$ sıfır ya da negatif çıkabilir. Örnek olarak metinde RARBER olsun: ARBER kısmı eşleşir, $k = 5$; uyuşmayan karakter R ve $t_1(R) = 3$. O zaman $3 - 5 = -2$ çıkar.

> **Ders sonrası not:** Hoca bu hesabı "3 eksi 5, 2 çıkacaktı" diye söyledi; doğrusu $3 - 5 = -2$'dir. Bu yüzden $d_1 = \max(-2, 1) = 1$ olur.

Negatif kaydırma örüntüyü sola götürmek demektir; daha önceki slaytlarda görülen "geri gelme" durumu budur ve anlamsızdır. Sıfır kaydırma ise örüntüyü hiç kaydırmadan devam etmek demektir; bu da bir seçenek değildir. En kötü ihtimalle 1 kaydırılır. Formüldeki değer 1'den büyükse o değer alınır.

8. Boyer-Moore: İyi Sonek Kaydırması

8.1 Fikir

BARBER örneğinde tekrar eden kısım yalnız tek harflerdir: bir R daha, bir B daha var. AAAB ya da ABABAB gibi uzun tekrarlı parçalar olmadığı için kaydırmayı yalnız kötü karakter üzerinden yapmak büyük bir sorun oluşturmaz. Fakat örüntüde tekrarlı parçalar varsa, kötü karakter ve iyi sonek kaydırmalarını birlikte değerlendirmek daha iyi sonuç verir.

İyi sonek kaydırmasında, sondan eşleşen k karakterlik sonekin (suffix) örüntü içinde bir önceki geçişine bakılır. Kaydırma, bu önceki geçişi metindeki eşleşmiş kısmın altına getirecek kadardır. Sonekin başka tam geçişi yoksa, sonekin bir parçasının örüntünün başıyla (önekiyle) hizalanmasına bakılır. Bu tablo da ön işlemde hesaplanır. Tablonun değerine d_2 denir.

8.2 ABCBAB Örneği

Slayttaki örüntü ABCBAB idi ($m = 6$).

- $k = 1$**, sonek `B`:** En sondaki `B`'den başka `B` var mı? Var; en yakını iki önceki konumda. $d_2 = 2$.
- $k = 2$**, sonek `AB`:** `AB` örüntünün başında da var ve `A` ile `A`, `B` ile `B` arasındaki uzaklık 4'tür. Hoca şöyle açıkladı: `B`'lerin eşleşmesi tek başına bir şey ifade etmiyor, çünkü daha uzun bir eşleşme (`AB`) var. `AB`'nin örüntüde tekrar ortaya çıkması ancak 4 kaydırmada mümkün. Kötü karakterden 6 kaydırmayı düşünüyor olsanız bile, `AB` eşleştiği ve `AB` örüntünün içinde geçtiği için ancak 4 kaydırabilirsiniz. $d_2 = 4$.
- $k = 3$**, sonek `BAB`:** `BAB` örüntüde başka yerde geçmiyor. Önceki `AB`'ye uzaklık hâlâ 4: `A` ile `A`, `B` ile `B` arası 4. Baştaki `B` kaydırıldığında karşılığında çakışan bir karakter kalmıyor. $d_2 = 4$.
- $k = 4$**, sonek `CBAB`:** $d_2 = 4$.
- $k = 5$**, sonek `BCBAB`:** $d_2 = 4$.

k	Sonek	d_2
1	`B`	2
2	`AB`	4
3	`BAB`	4
4	`CBAB`	4
5	`BCBAB`	4

Bu değerler Bölüm 13'teki programla doğrulandı.

Soru (hocanın): `C`'nin yerine `A` yazsaydık, yani örüntü `ABABAB` olsaydı, `AB` eşleşmesinde örüntüyü en fazla ne kadar kaydırabilirdik? **Cevap:** 2. `AB`'nin bir önceki `AB`'ye uzaklığı bu durumda 2 olur.

> **Ek bilgi (derste anlatılmadı):** Dersteki kural, sonekin bir önceki geçişine bakar. Kuralın daha güçlü bir biçiminde, önceki geçişin önündeki karakterin, son geçişin önündeki karakterden farklı olması da istenir; aynıysa o hizalamada aynı uyuşmazlık tekrar oluşur. `ABABAB` için `AB`'nin önceki geçişinin (konum 2) önünde de `B` vardır. Bu yüzden güçlü kural $k = 2$ için 4 verir (programla hesaplandı). Dersteki 2 değeri güvenlidir, yani eşleşme kaçırmaz, sadece daha az kaydırır. `ABCBAB` ve `BAOBAB` tablolarında iki kural aynı sonucu verir.

8.3 Önek ve Sonek

Hoca `ABCBAB` için önek (prefix) ve sonekleri (suffix) sordu. İyi sonek kaydırmasında sonekler, yani sağdan sola doğru büyüyen parçalar kullanılır.

- **Önekler:** `A`, `AB`, `ABC`, `ABCB`, `ABCBA`.
- **Sonekler:** `B`, `AB`, `BAB`, `CBAB`, `BCBAB`.

9. İki Kaydırmanın Birleştirilmesi

Birleştirilmiş Boyer-Moore'da iki denetim mekanizması vardır:

- Kötü karakter kaydırması, uyuşmazlıktan dolayı ne kadar kaydırılacağını hesaplar (d_1).
- İyi sonek kaydırması, örüntünün içindeki benzerlik bilgisinden ne kadar kaydırılacağını hesaplar (d_2).

Hoca, algoritmanın yalnız birini de gerçekleyebileceğinizi, bunun yanlış olmadığını söyledi. Ama ikisi birlikte kullanılınca örüntü daha hızlı hareket eder. Kural:

$$d = \begin{cases} d_1 & k = 0 \\ \max(d_1, d_2) & k > 0 \end{cases}$$

> **Hocanın vurgusu:** İki tablo da bir değer üretir; hangisi daha büyükse örüntü o kadar sağa kaydırılır. İki durumu birlikte irdelemek örüntüyü daha fazla hareket ettirme şansı verir. Bu önemli bir özelliktir.

Soru (hocanın): $k > 0$ olması ne anlama gelir? **Cevap (bir öğrenci):** Eşleşen eleman sayısı. Hoca ekledi: Hiç eşleşen eleman yoksa iyi sonek tarafında örüntüden kaynaklı kazanç sağlayacak bir tekrar hesabı da oluşamaz. Eşleşme olmadığında iyi sonek hesabı işe katılmaz; katılırsa yanıltıcı bir kaydırma olur ve hatalı sonuç üretir. Yani eşleşme yoksa d_1, eşleşme varsa ikisinin büyüğü kullanılır.

Algoritmanın adımları:

1. Kötü karakter tablosunu (t_1) oluştur.
2. İyi sonek tablosunu (d_2) oluştur.
3. Örüntüyü metnin başına hizala.
4. Sağdan sola karşılaştır. Bütün karakterler eşleşirse dur. Uyuşmazlık olursa k'ya göre d'yi hesapla ve örüntüyü d kadar sağa kaydır; 4. adımı tekrarla.

10. Tablet Üzerinde Örnek: BAOBAB

Hoca tableti bağlayıp ekranda bir örnek çözdü.

10.1 Kötü Karakter Tablosu

Örüntü `BAOBAB`, $m = 6$. Hoca öğrencilerle tabloyu doldurdu: `A`'ya 1, `B`'ye 2, `C`'ye 6, `D`'ye 6, `O`'ya 3.

c	A	B	O	Diğerleri (boşluk dahil)
$t_1(c)$	1	2	3	6

Bir öğrenci `A`'nın değerini "son elemana uzaklık" diye hesapladığını söyledi. Hoca, bilgisayarın tabloyu baştan sona doldurduğunu hatırlattı:

- `A`'nın ilk değeri 6'dır. Birinci `A` ($j = 1$) ile 4 olur, ikinci `A` ($j = 4$) ile 1 olur.
- `B`'nin ilk değeri 5'tir ($j = 0$), sonra `B` ($j = 3$) ile 2 olur. Son `B` ($j = 5$) hesaba katılmaz.
- `O` ($j = 2$): $6 - 1 - 2 = 3$.

10.2 İyi Sonek Tablosu

k	Sonek	d_2
1	`B`	2
2	`AB`	5
3	`BAB`	5
4	`OBAB`	5
5	`AOBAB`	5

- $k = 1$**:** Tek bir `B` eşleşmesi. Bir önceki `B` 2 geride; $d_2 = 2$.
- $k = 2$**:** `AB` örüntüde başka yerde geçmiyor. Öbür `B`'nin (konum 3) önünde `O` var; `OB` ile `AB` eşleşmediği için onu referans alamayız. Ancak örüntü kaydırılınca baştaki `B`, sonekin son harfi `B` ile hizalanabilir. Bunun uzaklığı 5'tir; $d_2 = 5$. Hoca bunu ekranda örüntüyü kırmızıyla 5 sağa kaydırarak gösterdi.
- $k = 3, 4, 5$**:** `BAB`, `OBAB`, `AOBAB` için de tekrar eden bir durum yok. Yine yalnız baştaki `B`, sonekin son `B`'siyle hizalanabilir. $d_2 = 5$.

Bu değerler programla doğrulandı (Bölüm 13).

10.3 Arama

Metin `BESS_KNEW_ABOUT_BAOBABS` (`_` boşluk karakteridir). Konumlar 0'dan sayılır.

Adım 1 (hiza 0): Örüntünün son harfi `B` ile metnin 5. karakteri `K` karşılaştırılır. Eşleşme yok, $k = 0$.

- $d_1 = \max(t_1(K) - 0, 1) = 6$.
- $k = 0$ olduğu için iyi sonekten hiçbir şey gelmez.
- $d = 6$. Örüntü 6 sağa kaydırılır.

Adım 2 (hiza 6): Metin parçası `NEW_AB`. `B` ve `A` eşleşir, sonra örüntüdeki `B` ile metindeki boşluk karşılaştırılır ve uyuşmaz. $k = 2$.

- Hoca önce $t_1(B) - 2 = 2 - 2 = 0$ yazdı ve max yüzünden 1 olacağını söyledi. Sonra öğrencilere hesapta bir sorun olup olmadığını sordu. Bir öğrenci "boşluk olmayacak mı?" diye uyardı.
- Doğrusu: uyuşmayan karakter metindeki boşluktur, $t_1(\text{boşluk}) = 6$. $d_1 = \max(6-2, 1) = 4$.
- $k \neq 0$ olduğu için d_2'ye bakılabilir. `AB` eşleşmesinin karşılığı $d_2 = 5$.
- $d = \max(4, 5) = 5$.

  Hocanın vurgusu: Bu hataya düşmemek gerekir. Formüldeki c metindeki eşleşmeyen karakterdir, örüntüdeki karakter değildir. Burada c `B` değil, boşluktur.

Adım 3 (hiza 11): Metin parçası `BOUT_B`. Son `B`'ler eşleşir, sonra örüntüdeki `A` ile metindeki boşluk uyuşmaz. $k = 1$.

- $d_1 = \max(6 - 1, 1) = 5$.
- $d_2 = 2$ (bir eşleşme).
- $d = \max(5, 2) = 5$.

Adım 4 (hiza 16): Metin parçası `BAOBAB`. Altı karakter de sağdan sola eşleşir. Örüntü 16. konumda bulunur.

Adım	Hiza	k	Uyuşmayan c	d_1	d_2	Kaydırma
1	0	0	`K`	6	kullanılmaz	6
2	6	2	boşluk	4	5	5
3	11	1	boşluk	5	2	5
4	16	6	—	—	—	bulundu

Bölüm 13'teki program aynı izi üretir: hiza 0, 6, 11 için kaydırmalar 6, 5, 5 ve sonuç konum 16. Aynı programdaki Horspool da örüntüyü 16. konumda bulur.

Hoca son olarak işin burada bitmek zorunda olmadığını söyledi. Metinde örüntünün başka geçişleri de aranıyorsa algoritma, eşleşme bulduktan sonra devam edecek şekilde değiştirilmelidir.

11. Rabin-Karp Algoritması

11.1 Parmak İzi Fikri

Önceki algoritmalar birebir karakter karşılaştırması yapıyordu. Rabin-Karp algoritması ise **parmak izi (fingerprint)** karşılaştırması yapar: tüm özellikler arasından ayırt edici olanlar seçilerek arama yapılır.

Hoca bunu görüntü işlemeden bir benzetmeyle anlattı. Yüz tanımada yüzde binlerce nokta vardır. Görüntü işlemede çok eski yaklaşımlardan biri olan şablon eşleme (template matching), naif metin aramadaki mantıkla piksel piksel eşleşme kontrol eder ve aday resmin aynı olup olmadığını ya da ne kadar benzediğini hesaplar. Bütün pikselleri kontrol etmek akıllıca değildir. Bunun yerine çeşitli çekirdekler (kernel) gezdirilip en büyük havuzlama (max pooling) gibi işlemlerle temel özellikler çıkarılır ve resmin parmak izi denen ayırt edici noktalar üzerinden eşleşme kontrol edilir. Rabin-Karp'ta da benzer bir mantık vardır.

11.2 Sayılarla Örnek

Mantığı anlatmak için slaytta metin ve örüntü rakamlarla verilmişti. Karakterlerle çalışırken önce dönüşüm yapmak gerekir.

- Örüntü: `26535` ($m = 5$). Metin 3 ile başlayıp 3 ile biten bir rakam dizisi.

- Asal sayı olarak 997 seçilmiş.
- Örüntünün değeri: $26535 \bmod 997 = 613$.
- Metnin ilk 5 haneli penceresi: $31415 \bmod 997 = 508$.

Değerleri doğrulayalım (ara adım eklendi): $997 \cdot 26 = 25922$ ve $26535 - 25922 = 613$; $997 \cdot 31 = 30907$ ve $31415 - 30907 = 508$.

Değerler eşit değilse ($508 \neq 613$) örüntü bir sağa kaydırılır ve bir sonraki pencerenin değeri hesaplanır. Böylece her konumda beş rakamın ayrı ayrı karşılaştırılması yerine tek bir değer karşılaştırması yapılır. Hoca, kaydırmanın yine birer birer yapıldığını, naif aramadan farkın kaydırma değil karşılaştırma sayısının azalması olduğunu belirtti.

Değerler eşit çıkarsa bu kesin bir eşleşme değildir. 26535 yerine başka bir sayının da 997'ye göre kalanı 613 olabilir. Bu yüzden kalan eşleşmesi tespit edildikten sonra rakamlar tek tek karşılaştırılır.

> **Hocanın vurgusu:** Önce modüler eşleşmenin sağlandığı tespit edilir, sonra rakamların birebir eşleşip eşleşmediği kontrol edilir. Kalanın eşit olması tek başına garanti değildir.

11.3 Özet (Hash) Değeri

Rabin-Karp aslında bir özet (hash) değeri hesaplar. Hoca hash'in arama tekniklerinde $O(1)$ karmaşıklıklı aramada ve siber güvenlikte, şifreleme ve kriptolojide kullanıldığını söyledi. Temel fikir bir metin grubunun ya da sayının özet değerini üretmektir. Bu değer genelde modüler aritmetikle hesaplanır; temel yöntemler bölme (division) ve çarpma (multiplication) yöntemleridir. Ayrıntısı sonraki dönem Algoritma Analizi dersinde görülecektir.

Slayttaki maddeler:

- Özet değerleri eşit değilse sonraki m karakterlik dizinin özet değeri hesaplanır.
- Özet değerleri eşitse, kaba kuvvet karşılaştırmasıyla örüntü ve dizideki m karakter tek tek karşılaştırılır. Bütün karakterler tutarsa örüntü bulunmuştur.

11.4 Büyük Sayılar ve Kayan Hesap

Örüntü uzadıkça sayılar büyür. Bu yüzden hesabı sayıyı küçülterek yapan teknikler kullanılır. Slaytta örüntünün değeri basamak basamak hesaplanıyordu: önce 2, sonra 26, sonra 265 ... ve her adımda mod alınıyordu. Adımları tamamlayalım (ara adım eklendi):

Adım	Önceki değer $\cdot 10$ + rakam	mod 997
2	2	2
6	$2 \cdot 10 + 6 = 26$	26
5	$26 \cdot 10 + 5 = 265$	265
3	$265 \cdot 10 + 3 = 2653$	659
5	$659 \cdot 10 + 5 = 6595$	613

Sonuç yine 613'tür. Sayı $2 \cdot 10000 + 6 \cdot 1000 + 5 \cdot 100 + 3 \cdot 10 + 5$ gibi kuvvetlerle de yazılabilir. Ama her seferinde 10000 ile çarpmak yerine 10 ile çarpıp yeni rakamı eklemek işi kısaltır.

Ardışık pencerelerde de aynı fikir kullanılır (kayan özet, rolling hash). 41592 hesaplandıktan sonra 15926 baştan hesaplanmaz:

1. Baştaki rakamın katkısı çıkarılır: $41592 - 4 \cdot 10000 = 1592$.
2. 10 ile çarpılır: $1592 \cdot 10 = 15920$.
3. Yeni rakam eklenir: $15920 + 6 = 15926$.

Böylece çarpma ve bölmenin az, toplama ve çıkarmanın ağırlıkta olduğu bir yöntemle modüler aritmetik işlemleri daha verimli yapılır. Hoca bunun ayrıntısı üzerinde durmadı, kendiniz inceleyebileceğinizi söyledi.

11.5 Karakterlere Geçiş

Rakamların yerine harfler kullanıldığında harflerin ASCII tablosundaki karşılıkları alınır, sayıya çevrilir ve aynı modüler aritmetik uygulanır. Hoca, Horner kuralındaki gibi her harfi farklı bir R^i katsayısıyla çarpmanın, `AB` ile `BA` gibi dizilerin aynı değeri üretmesini önlemek, yani bu olasılığı düşürmek için uygulanan tekniklerden biri olduğunu söyledi.

> **Ek bilgi (derste anlatılmadı):** Özet karşılaştırması bir konumu $O(1)$ işlemle eler. Ama her eşit özet için m karakterlik doğrulama gerektiğinden, çok sayıda çakışma olursa en kötü durum yine $O(nm)$ olur. Mod olarak büyük bir asal seçmek çakışma sayısını azaltır. Bölüm 13'teki örnekte metindeki 12 pencereden yalnız biri 613 kalanını verir ve o da gerçek eşleşmedir (programla hesaplandı).

12. Karmaşıklık Karşılaştırması ve Deneysel Sonuçlar

12.1 Tablo

N karakterli metinde M karakterli örüntü aramanın maliyeti:

Algoritma	Tipik	En kötü
Kaba kuvvet	$1.1N$ karakter karşılaştırması	MN karakter karşılaştırması
Karp-Rabin	$3N$ aritmetik işlem	$3N$ aritmetik işlem
KMP	$1.1N$ karakter karşılaştırması	$2N$ karakter karşılaştırması
Boyer-Moore	yaklaşık N/M karakter karşılaştırması	$3N$ karakter karşılaştırması

Hocanın yorumu:

- KMP, Boyer-Moore'un sağdan sola yaptığı eşleşme kontrolünü soldan sağa yapar. Ama kullandığı mantık Boyer-Moore kadar gelişmiş değildir; bu yüzden tipik durumda büyük bir iyileşme sağlamaz.
- Boyer-Moore, tekrar etmeyen örüntülerin olduğu standart bir yapıda N/M'lik karşılaştırma yapar. Diğerleri $1.1N$'lik karşılaştırma yapar. Özel durumlar dışında Boyer-Moore çok daha etkilidir.
- En kötü durumda kaba kuvvet oldukça kötüdür: $N \cdot M$.

> **Ders sonrası not:** Hoca en kötü durum için Karp-Rabin ve Boyer-Moore'un "bir tık daha iyi" olduğunu, KMP'nin de "hemen hemen aynı" karmaşıklığa sahip olduğunu söyledi. Tabloya göre KMP'nin en kötü durumu ($2N$) Boyer-Moore'unkinden ($3N$) iyidir. Boyer-Moore'un asıl üstünlüğü tipik durumdadır (N/M).

> **Ek bilgi (derste anlatılmadı):** Tablodaki Boyer-Moore $3N$ sınırı, iki kuralın (kötü karakter ve güçlü iyi sonek) birlikte kullanıldığı ve örüntünün metinde geçmediği durum içindir. Örüntünün bütün geçişleri aranırken temel Boyer-Moore en kötü durumda $O(nm)$'ye çıkabilir; Galil kuralı gibi iyileştirmeler bunu önler. Yalnız kötü karakter kuralını kullanan sürüm (Horspool) en kötü durumda da $O(nm)$'dir.

12.2 Deneysel Grafikler

Son olarak algoritmaların deneysel performansları gösterildi:

- Örüntü büyüdükçe naif yöntemin performansında çok büyük bir değişiklik olmuyor. KMP için de aynı şey geçerli.
- Belli bir noktadan sonra naif yöntem ve KMP, Boyer-Moore'un altında kalıyor.
- Boyer-Moore'da örüntünün büyümesi aramada performansı çok daha fazla artırıyor.
- Grafikte araları açık gibi görünse de, bir makaledeki deneylere göre Boyer-Moore ile Boyer-Moore-Horspool özellikle örüntü büyüdükçe birbirine oldukça yakın sonuçlar üretiyor. Bir noktadan sonra paralel seyrediyorlar.

Hocanın vurgusu: Boyer-Moore ile Horspool'un sonuçları birbirine çok yakın olduğu için, gerçeklemesi kolay olan Horspool tercih edilebilir.

Bu algoritmaların daha farklı, özelleştirilmiş sürümleri ve performans iyileştirmeleri de vardır. Hoca, standart bir iş yapan biri için bu algoritmaların metin aramada tatmin edici sonuçlar ürettiğini söyledi.

13. Algoritmaların C ile Gerçeklemesi

Derste C kodu olarak yalnız kaba kuvvet `search` fonksiyonu gösterildi. Horspool ve Boyer-Moore için sözde kod ile kurallar verildi, Rabin-Karp ise adımlarıyla anlatıldı. Aşağıdaki program bu anlatımın C'ye aktarılmış hâlidir ve dersteki örnekleri çalıştırır. İyi sonek tablosu dersteki kurala göre (sonekin bir önceki geçişi, yoksa önekle hizalama) doğrudan tanımdan hesaplanır. Program `gcc -Wall` ile uyarısız derlendi.

```c
#include <stdio.h>
#include <string.h>

#define ALFABE 256

/* Kaba kuvvet: bulunamazsa N döner (slayttaki gibi) */
int kaba_kuvvet(const char *pat, const char *txt, long *karsilastirma) {
    int M = strlen(pat);
    int N = strlen(txt);
    int i, j;
    for (i = 0; i <= N - M; i++) {
        for (j = 0; j < M; j++) {
            (*karsilastirma)++;
            if (txt[i + j] != pat[j])
                break;
        }
        if (j == M)
            return i;
    }
    return N;
}

/* Horspool kaydırma tablosu (= Boyer-Moore t1 tablosu) */
void kaydirma_tablosu(const char *P, int tablo[ALFABE]) {
    int m = strlen(P);
    for (int i = 0; i < ALFABE; i++)
        tablo[i] = m;
    for (int j = 0; j <= m - 2; j++)          /* son karakter hariç */
        tablo[(unsigned char)P[j]] = m - 1 - j;
}

int horspool(const char *P, const char *T) {
    int tablo[ALFABE];
    int m = strlen(P), n = strlen(T);
    kaydirma_tablosu(P, tablo);
    int i = m - 1;                            /* örüntünün sağ ucu */
    while (i <= n - 1) {
        int k = 0;
        while (k <= m - 1 && P[m - 1 - k] == T[i - k])
            k++;
        if (k == m)
            return i - m + 1;
        i += tablo[(unsigned char)T[i]];
    }
    return -1;
}
```

```c
/* İyi sonek tablosu: d2[k], k = 1..m-1 (derste anlatılan kural) */
void iyi_sonek_tablosu(const char *P, int d2[]) {
    int m = strlen(P);
    for (int k = 1; k <= m - 1; k++) {
        int s;
        for (s = 1; s < m; s++) {
            /* örüntü s sağa kaydırıldığında eşleşen k'lık sonekin
               altına gelen kısım soneke uyuyor mu? Örüntünün başı
               dışarı taşarsa yalnız önek kısmı karşılaştırılır. */
            int ok = 1;
            for (int t = 0; t < k; t++) {
                int p = m - 1 - t - s;        /* kaymış örüntüdeki konum */
                if (p < 0) break;
                if (P[p] != P[m - 1 - t]) { ok = 0; break; }
            }
            if (ok) break;
        }
        d2[k] = s;
    }
}

int boyer_moore(const char *P, const char *T, int iz) {
    int t1[ALFABE], d2[64];
    int m = strlen(P), n = strlen(T);
    kaydirma_tablosu(P, t1);
    iyi_sonek_tablosu(P, d2);
    int i = m - 1;
    while (i <= n - 1) {
        int k = 0;
        while (k <= m - 1 && P[m - 1 - k] == T[i - k])
            k++;
        if (k == m)
            return i - m + 1;
        char c = T[i - k];                    /* metindeki uyuşmayan karakter */
        int d1 = t1[(unsigned char)c] - k;
        if (d1 < 1) d1 = 1;
        int d = (k == 0) ? d1 : (d1 > d2[k] ? d1 : d2[k]);
        if (iz)
            printf("  hiza=%2d k=%d c='%c' d1=%d d2=%d d=%d\n",
                   i - m + 1, k, c, d1, k ? d2[k] : 0, d);
        i += d;
    }
    return -1;
}

/* Rabin-Karp (rakam dizisi, taban 10, mod q) */
int rabin_karp(const char *pat, const char *txt, long q) {
    int M = strlen(pat), N = strlen(txt);
    long R = 10, RM = 1, ph = 0, th = 0;
    for (int i = 1; i <= M - 1; i++) RM = (R * RM) % q;
    for (int j = 0; j < M; j++) {
        ph = (R * ph + (pat[j] - '0')) % q;
        th = (R * th + (txt[j] - '0')) % q;
    }
    for (int i = 0; ; i++) {
        if (ph == th && strncmp(pat, txt + i, M) == 0)
            return i;
        if (i + M >= N) break;
        th = (th + q - RM * (txt[i] - '0') % q) % q;
```

```
        th = (th * R + (txt[i + M] - '0')) % q;
    }
    return N;
}

int main(void) {
    long kk = 0;
    int r = kaba_kuvvet("AAAAB", "AAAAAAAAAB", &kk);
    printf("Kaba kuvvet AAAAB/AAAAAAAAAB: konum=%d karsilastirma=%ld\n", r, kk);

    int t[ALFABE];
    kaydirma_tablosu("BARBER", t);
    printf("BARBER t1: A=%d B=%d E=%d R=%d S=%d\n", t['A'], t['B'], t['E'],
           t['R'], t['S']);
    kaydirma_tablosu("BAOBAB", t);
    printf("BAOBAB t1: A=%d B=%d O=%d _=%d\n", t['A'], t['B'], t['O'], t['_']);

    int d2[64];
    const char *ps[] = {"ABCBAB", "ABABAB", "BAOBAB"};
    for (int p = 0; p < 3; p++) {
        iyi_sonek_tablosu(ps[p], d2);
        printf("%s d2:", ps[p]);
        for (int k = 1; k < 6; k++) printf(" %d", d2[k]);
        printf("\n");
    }

    const char *metin = "BESS_KNEW_ABOUT_BAOBABS";
    printf("Boyer-Moore izi:\n");
    printf("BM sonuc=%d\n", boyer_moore("BAOBAB", metin, 1));
    printf("Horspool sonuc=%d\n", horspool("BAOBAB", metin));
    printf("Rabin-Karp 26535=%d\n", rabin_karp("26535", "3141592653589793", 997));
    return 0;
}
```

Çıktı:

```
Kaba kuvvet AAAAB/AAAAAAAAAB: konum=5 karsilastirma=30
BARBER t1: A=4 B=2 E=1 R=3 S=6
BAOBAB t1: A=1 B=2 O=3 _=6
ABCBAB d2: 2 4 4 4 4
ABABAB d2: 2 2 2 2 2
BAOBAB d2: 2 5 5 5 5
Boyer-Moore izi:
  hiza= 0 k=0 c='K' d1=6 d2=0 d=6
  hiza= 6 k=2 c='_' d1=4 d2=5 d=5
  hiza=11 k=1 c='_' d1=5 d2=2 d=5
BM sonuc=16
Horspool sonuc=16
Rabin-Karp 26535=6
```

Çıktıdaki her satır derste bulunan değerle aynıdır: kaba kuvvetin en kötü durum sayımı (30), `BARBER` ve `BAOBAB` kaydırma tabloları, `ABCBAB` ve `BAOBAB` iyi sonek tabloları, `ABABAB` için $k = 2$'de 2 değeri ve BAOBAB aramasının izi. İzdeki ilk satırda $k = 0$ olduğu için d_2 kullanılmaz; programda 0 olarak basılır. Bu programdaki Horspool ve kaba kuvvet farklı "bulunamadı" değerleri döndürür: sözde koddaki gibi -1 ve slayttaki C kodundaki gibi N.

Örnek Sorular

Bu dersin hazır karesi yoktur. Aşağıdaki sorular derste sözlü sorulan ya da tablette birlikte çözülen sorulardır; metinleri altyazıdan kurulmuştur.

Soru 1 (derste sözlü, 00:18:26) Kaba kuvvette örüntüyü birer birer kaydırmak maliyetli görünüyor. `ABRA` aranırken ilk karakterler eşleşip sonra uyuşmazlık çıkarsa örüntüyü nereye kaydırırdınız?

> **Çözüm (derste verildi):** Örüntü `B`'nin altına kaydırılırsa eşleşme ihtimali yoktur. Örüntü metindeki bir sonraki `A`'nın altına kaydırılmalıdır. Genel kural: mümkünse örüntü boyu kadar, değilse eşleşmenin sağlanabileceği ilk noktaya kadar kaydırmak.

Soru 2 (derste sözlü, 00:26:39) `BARBER` aranırken son karakter hizasındaki metin karakteri `S` ve `R` ile eşleşmiyor. Örüntüyü neden gönül rahatlığıyla 6 kaydırabiliriz?

> **Çözüm (derste verildi):** `S`, `BARBER` içinde geçmez. Örüntüde onunla hizalanacak bir harf olmadığı için örüntü boyu kadar, 6 kaydırılır.

Soru 3 (derste sözlü, 00:28:23) Son karakter hizasındaki metin karakteri `B` ise `BARBER` ne kadar kaydırılır? Neden ilk (sağdaki) `B` ile hizalanır? Karakter `R` ya da `A` olsaydı?

> **Çözüm (derste verildi):** Sağdan en yakın `B`, sondan 2 önceki konumdadır; kaydırma 2. `R` olsaydı 3, `A` olsaydı 4. Hizalamada harfin örüntüdeki sağdan en yakın geçişi alınır.

Soru 4 (derste sözlü, 00:30:56) `LEADER` örüntüsünde `R` ve `E` eşleşmiş, `M` ile `D` eşleşmemiş. Örüntünün komple kaydırılmasının temel sebebi nedir; bunu algoritmaya nasıl tarif ederiz?

> **Çözüm (derste verildi):** Kaydırma son karakter hizasındaki `R`'ye göre belirlenir. Örüntünün ilk beş karakterinde başka `R` olmadığı için `R` ile tekrar hizalanma ihtimali yoktur; kaydırma tablodaki değer, yani $m = 6$'dır.

Soru 5 (derste sözlü, 00:34:02) `BARBER` kaydırma tablosundaki 4, 2, 6, 6, 1, 6 değerleri neye göre elde edilir? Tablonun başlangıç değeri ne olmalıdır? İlk `B` görüldüğünde tabloya ne yazılır?

> **Çözüm (derste verildi):** Başlangıç değeri örüntü uzunluğudur ($m = 6$). Örüntü soldan sağa, son karakter hariç gezilir ve her karaktere $m - 1 - j$ yazılır. İlk `B` görüldüğünde 5 yazılır; ikinci `B`'de 2 ile güncellenir. `A` 4, `R` 3, `E` 1 olur. Son `R` işlenmez, değeri 3 kalır.

Soru 6 (derste sözlü, 01:01:25) `BARBER` için $t_1(S) - 2 = 4$ ve $t_1(A) - 2 = 2$ hesaplarını sözle nasıl ifade edersiniz?

> **Çözüm (derste verildi):** Tablodaki kaydırma değerinden eşleşen karakter sayısı k çıkarılır. `S` örüntüde olmadığı için tabloda 6'dır; iki karakter eşleştiği için $6 - 2 = 4$ kaydırılır. `A` için $4 - 2 = 2$. Genel formül $d_1 = \max(t_1(c) - k, 1)$.

Soru 7 (derste sözlü, 01:04:25) Kötü karakter formülündeki "virgül 1", yani $\max(\dots, 1)$ neden vardır?

> **Çözüm (derste verildi):** $t_1(c) - k$ sıfır ya da negatif çıkabilir. Örneğin `RARBER` metninde $k = 5$, $t_1(R) = 3$: $3 - 5 = -2$. Sola kaydırma ya da hiç kaydırmama anlamsızdır; en az 1 kaydırılır.

Soru 8 (derste sözlü, 01:08:59) `ABCBAB` örüntüsünde `C`'nin yerine `A` yazsaydık (`ABABAB`), `AB` eşleştiğinde örüntüyü en fazla ne kadar kaydırabilirdik?

> **Çözüm (derste verildi):** 2. `AB`'nin bir önceki `AB`'ye uzaklığı 2 olur. Program dersteki kuralla aynı değeri verir (`ABABAB d2: 2 2 2 2 2`). Daha güçlü iyi sonek kuralının 4 verdiği Bölüm 8.2'deki ek bilgi bloğunda açıklandı.

Soru 9 (derste sözlü, 01:10:29) `ABCBAB` dizisinin önekleri ve sonekleri nelerdir?

> **Çözüm (derste kısmen verildi, liste tanımdan tamamlandı):** Önekler `A`, `AB`, `ABC`, `ABCB`, `ABCBA`. Sonekler `B`, `AB`, `BAB`, `CBAB`, `BCBAB`.

Soru 10 (derste sözlü, 01:15:07) Birleştirilmiş kuralda $k > 0$ olması ne anlama gelir? $k = 0$ iken neden yalnız d_1 kullanılır?

> **Çözüm (derste verildi):** k eşleşen eleman sayısıdır. Eşleşme yoksa iyi sonek tarafında kazanç sağlayacak bir tekrar hesabı oluşamaz; iyi sonek katılırsa yanıltıcı kaydırma olur. Bu yüzden $k = 0$ iken $d = d_1$, $k > 0$ iken $d = \max(d_1, d_2)$.

Soru 11 (derste sözlü, tablette, 01:20:41) `BAOBAB` için kötü karakter tablosunu doldurun. `A` ve `B`'nin değerleri tablo doldurulurken hangi ara değerlerden geçer?

Çözüm (derste verildi): `A` = 1, `B` = 2, `O` = 3, diğer bütün karakterler (boşluk dahil) 6. `A` önce 6, sonra 4, sonra 1 olur. `B` önce 6, sonra 5, sonra 2 olur; son `B` işlenmez.

Soru 12 (derste sözlü, tablette, 01:23:41) `BAOBAB` için iki karakter (`AB`) eşleştiğinde iyi sonek kaydırması kaç olur?

Çözüm (derste verildi): Öğrencilerden 6 ve 4 cevapları geldi; ikisi de yanlıştır. `AB` örüntüde başka yerde geçmez; konum 3'teki `B`'nin önünde `O` vardır, `OB` ile `AB` uyuşmaz. Yalnız baştaki `B`, sonekin son `B`'siyle hizalanabilir; uzaklık 5. $d_2 = 5$. Program aynı değeri verir.

Soru 13 (derste sözlü, tablette, 01:28:46) `BESS_KNEW_ABOUT_BAOBABS` metninde `BAOBAB` örüntüsünü Boyer-Moore ile arayın. Her adımda d_1, d_2 ve kaydırmayı bulun.

Çözüm (derste verildi): Adım 1: $k = 0$, c = `K`, $d_1 = 6$, $d = 6$. Adım 2: $k = 2$, c = boşluk, $d_1 = 6 - 2 = 4$, $d_2 = 5$, $d = 5$. Adım 3: $k = 1$, c = boşluk, $d_1 = 6 - 1 = 5$, $d_2 = 2$, $d = 5$. Adım 4: tam eşleşme, konum 16. Hoca 2. adımda önce örüntüdeki `B`'yi alıp $2 - 2 = 0$ yazdı; bir öğrencinin uyarısıyla uyuşmayan karakterin metindeki boşluk olduğu düzeltildi. Program aynı izi verir (Bölüm 13).

Dersin Toparlaması

Bu derste metin arama problemi ele alındı. Kaba kuvvet yöntemi örüntüyü her uyuşmazlıkta bir sağa kaydırır; tekrarlı örüntü ve metinlerde en kötü durumda $O(nm)$ karşılaştırma yapar. Daha akıllı yöntemlerin ortak fikri, uyuşmazlıkta örüntüyü eşleşme ihtimalinin olmadığı konumları atlayacak kadar kaydırmaktır. Bunun için örüntü üzerinde ucuz bir ön işlem yapılır. Boyer-Moore-Horspool, son karakter hizasındaki metin karakterine göre tek bir kaydırma tablosu kullanır. Tam Boyer-Moore, metindeki uyuşmayan karaktere göre kötü karakter kaydırmasını ($d_1 = \max(t_1(c) - k, 1)$) ve eşleşen sonekin tekrarına göre iyi sonek kaydırmasını (d_2) birlikte kullanır; $k > 0$ ise ikisinin büyüğü seçilir. BAOBAB örneğinde örüntü üç kaydırmada (6, 5, 5) bulundu.

Rabin-Karp ise karakterleri tek tek karşılaştırmak yerine pencerelerin özet değerlerini karşılaştırır. Kayan hesapla her yeni pencerenin değeri bir öncekinden bulunur, özet eşit çıkınca karakterler tek tek doğrulanır. Karşılaştırma tablosu ve deneysel grafikler, tipik durumda Boyer-Moore'un N/M civarında karşılaştırmayla en iyi sonucu verdiğini ve örüntü uzadıkça hızlandığını gösterdi. Horspool'un sonuçları Boyer-Moore'a çok yakın olduğu için, kolay gerçeklenmesi nedeniyle tercih edilebilir. Hoca, özet (hash) fonksiyonlarının ayrıntısının sonraki dönem Algoritma Analizi dersinde görüleceğini, önümüzdeki hafta ise sıralama algoritmalarının karşılaştırmalı olarak değerlendirileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.